03. Station 2: Feature Engine & JA4 Fingerprinting (`ja4.py`)

1. Overview & Purpose

`ja4.py` is the **core feature extraction engine** of the network sensor. It performs five critical security tasks:

1. **Metadata Normalization:** Extracts dual-stack IP addresses, ports, TCP flags, packet size, and hop limits into a unified dictionary.
2. **TCP MTU Reassembly:** Overcomes network MTU fragmentation when large modern TLS handshakes (e.g. Kyber/ML-KEM post-quantum ciphers) are split across multiple TCP segments.
3. **RFC 8701 GREASE Filtering:** Strips browser-injected randomness (GREASE ciphers/extensions) so identical client software always produces the exact same hash.
4. **JA4 Client Fingerprinting:** Calculates the 36-character standard JA4 signature (`t13d..._..._...`).
5. **Bidirectional Stateful Flow Caching:** Remembers the negotiated JA4 fingerprint and SNI in an in-memory session table keyed by `sorted((src_ip, src_port), (dst_ip, dst_port))` so encrypted packets that follow inherit the fingerprint in $O(1)$O(1) time.

2. The JA4 Fingerprint Anatomy

The JA4 fingerprint string consists of three parts separated by underscores:

$$\text{JA4} = \text{Prefix}_{\text{10 chars}} _ \text{CipherHash}_{\text{12 chars}} _ \text{ExtensionHash}_{\text{12 chars}}$$

JA4 = Prefix10 chars _ CipherHash12 chars _ ExtensionHash12 chars

Example: `t13d9013h2_c6771aded2ed_7b6eb23b1f16`

```
  t    13     d    90     13     h2    _  c6771aded2ed  _  7b6eb23b1f16
  |    |      |    |      |      |            |                 |
  |    |      |    |      |      |            |                 └─ SHA256[:12] of sorted
  |    |      |    |      |      |            |                    extensions & sig
algorithms
  |    |      |    |      |      |            └─ SHA256[:12] of sorted non-GREASE ciphers
  |    |      |    |      |      └─ First & last char of ALPN (e.g., "h2" for HTTP/2)
  |    |      |    |      └─ Count of clean TLS extensions (2 digits, max 99)
  |    |      |    └─ Count of clean cipher suites (2 digits, max 99)
  |    |      └─ SNI destination flag ("d" if domain requested, "i" if raw IP)
  |    └─ TLS Version ("13" for TLS 1.3, "12" for TLS 1.2, "00" for older/unknown)
  └─ Protocol transport ("t" for TCP, "q" for QUIC/UDP)
```

3. Function-by-Function Breakdown

1. `metadata(packet)`

- **Role:** Extracts L3 and L4 attributes into `master_directory`.
- **Dual-Stack Logic:**
  - If IPv4: extracts `packet["IP"].src`, `packet["IP"].dst`, and `packet["IP"].ttl`.
  - If IPv6: extracts `packet["IPv6"].src`, `packet["IPv6"].dst`, and `packet["IPv6"].hlim` (Hop Limit replaces TTL in IPv6).
- **TCP Layer:** Extracts `sport`, `dport`, `flags` (e.g. `S` for SYN, `SA` for SYN-ACK, `A` for ACK, `PA` for Push-ACK), and total packet length.

2. `ja4(packet)`

- **Role:** Direct Scapy TLS dissection parser.
- **Execution:** Used if Scapy natively parses `TLSClientHello` on single-segment packets. Extracts ciphers, extensions, ALPN (Extension 16), SNI (Extension 0), and Supported Versions (Extension 43).

3. `try_get_hello_bytes(packet, session_key)`

- **Role:** Handles TCP MTU fragmentation.
- **Why it is needed:** When a browser sends a TLS ClientHello declaring e.g. 1,693 bytes, it exceeds the standard Ethernet MTU (1,460 bytes TCP payload). The OS splits it into Segment 1 (1460

bytes) and Segment 2 (233 bytes). Scapy cannot dissect incomplete segments and marks them as `Padding`.

- **Mechanism:** Looks for the TLS Handshake byte (`0x16`) and ClientHello byte (`0x01`). Reads bytes 3-4 to calculate declared total length (`record_len + 5`). Buffers Segment 1 in `hello_buffer[session_key]` and concatenates Segment 2 on arrival, returning the complete reconstructed byte stream.

4. `ja4_from_raw(data)`

- **Role:** High-speed binary parser using byte slicing.
- **Execution:** Dissects raw reassembled bytes: parses Cipher Suites, reads Extension records, extracts SNI domain text, ALPN strings, Supported Versions, and Signature Algorithms without relying on third-party dissection.

5. `compute_ja4(clean_ciphers, clean_extensions, versions, sni, alpn, sni_hostname, sig_algs)`

- **Role:** Formulates the JA4 string.
- **Algorithm:**
  1. Formulates the 10-character prefix: `f"t{versions}{sni}{c_count}{e_count}{alpn}"`.
  2. Hex-encodes and alphabetically sorts cipher suites (e.g., `["002f", "0035", "1301"]`), joins with commas, hashes with SHA-256, and takes the first 12 characters.
  3. Filters out Extension 0 (SNI) and Extension 16 (ALPN) per JA4 specification, sorts extension hex values, appends sorted signature algorithms, hashes with SHA-256, and takes the first 12 characters.
  4. Returns `f"{prefix}_{cipher_hash}_{ext_hash}"` and `sni_hostname`.

6. `process_packet(packet)`

- **Role:** Pipeline coordinator for each packet.
- **Execution:**
  1. Invokes Gatekeeper `capture(packet)`. If `None`, aborts immediately.
  2. Extracts metadata into `meta`.
  3. Generates bidirectional `session_key = tuple(sorted([(src_ip, sport), (dst_ip, dport)]))`.
  4. Attempts Scapy parsing (`ja4()`), falling back to raw reassembly (`try_get_hello_bytes()` + `ja4_from_raw()`).
  5. If ClientHello is parsed, updates `session_table[session_key] = (ja4, sni)`.

6. If subsequent encrypted packet, performs $O(1)$O(1) lookup in `session_table` to inherit the connection's JA4 and SNI.
7. Returns the enriched `meta` dictionary.

4. Complete Source Code

```
python
from scapy.all import sniff, IP, TCP, IPv6
from scapy.layers.tls.handshake import TLSClientHello
from scapy.layers.tls.record import TLS
import hashlib
from capture import capture

session_table = {}
hello_buffer = {}

GREASE_TABLE = {
    0x0a0a, 0x1a1a, 0x2a2a, 0x3a3a,
    0x4a4a, 0x5a5a, 0x6a6a, 0x7a7a,
    0x8a8a, 0x9a9a, 0xaaaa, 0xbaba,
    0xcaca, 0xdada, 0xeaea, 0xfafa
}

def metadata(packet):
    if packet.haslayer(IP):
        source_ip = packet["IP"].src
        destination_ip = packet["IP"].dst
        time_to_live = packet["IP"].ttl
    elif packet.haslayer(IPv6):
        source_ip = packet["IPv6"].src
        destination_ip = packet["IPv6"].dst
        time_to_live = packet["IPv6"].hlim
    source_port = packet["TCP"].sport
    destination_port = packet["TCP"].dport
    flags = str(packet["TCP"].flags)
    packet_size = len(packet)
    master_directory = {
        "source_type": "network",
        "protocol": "tcp",
        "source_ip": source_ip,
        "dest_ip": destination_ip,
        "source_port": source_port,
        "dest_port": destination_port,
        "tcp_flags": flags,
        "packet_size": packet_size,
        "ja4": None,
        "event_type": "TCP",
        "metadata": {
            "time_to_live": time_to_live,
```

```
            "sni": None
        }
    }
    return master_directory

def ja4(packet):
    if not packet.haslayer(TLSClientHello):
        return None, None
    hello = packet[TLSClientHello]
    ciphers = hello.ciphers
    clean_ciphers = [c for c in ciphers if c not in GREASE_TABLE]
    clean_extensions = []
    extensions = hello.ext
    for e in extensions:
        if e.type not in GREASE_TABLE:
            clean_extensions.append(e.type)
    c_count = f"{min(len(clean_ciphers), 99):02d}"
    e_count = f"{min(len(clean_extensions), 99):02d}"
    sni = "i"
    sni_hostname = None
    versions = "00"
    alpn = "00"
    for e in extensions:
        if e.type == 0:
            sni = "d"
            if hasattr(e, "servernames") and len(e.servernames) > 0:
                sni_hostname = e.servernames[0].servername.decode("utf-8", errors="ignore")
        if e.type == 16:
            if hasattr(e, "protocols") and len(e.protocols) > 0:
                raw_protocol = e.protocols[0].protocol.decode("utf-8", errors="ignore")
                alpn = raw_protocol[0] + raw_protocol[-1]
        if e.type == 43 and hasattr(e, "versions"):
            if 0x0304 in e.versions:
                versions = "13"
            elif 0x0303 in e.versions:
                versions = "12"
    if versions == "00":
        if hello.version == 0x0303:
            versions = "12"
    return compute_ja4(clean_ciphers, clean_extensions, versions, sni, alpn, sni_hostname)

def try_get_hello_bytes(packet, session_key):
    if not packet.haslayer(TCP):
        return None
    payload = bytes(packet[TCP].payload)
    if len(payload) < 1:
```

```
            return None
        if session_key in hello_buffer:
            buf = hello_buffer[session_key]
            combined = buf["data"] + payload
            if len(combined) >= buf["needed"]:
                del hello_buffer[session_key]
                return bytes(combined[:buf["needed"]])
            else:
                buf["data"] = combined
                return None
        if len(payload) < 6:
            return None
        if payload[0] != 0x16 or payload[5] != 0x01:
            return None
        record_len = int.from_bytes(payload[3:5], "big")
        needed = record_len + 5
        if len(payload) >= needed:
            return payload[:needed]
        else:
            hello_buffer[session_key] = {"needed": needed, "data": payload}
            return None

    def ja4_from_raw(data):
        try:
            if len(data) < 43 or data[0] != 0x16 or data[5] != 0x01:
                return None, None
            client_version = int.from_bytes(data[9:11], "big")
            sess_id_len = data[43]
            pos = 44 + sess_id_len
            if pos + 2 > len(data):
                return None, None
            ciphers_len = int.from_bytes(data[pos:pos + 2], "big")
            pos += 2
            ciphers = []
            for i in range(0, ciphers_len, 2):
                if pos + i + 2 <= len(data):
                    c = int.from_bytes(data[pos + i:pos + i + 2], "big")
                    ciphers.append(c)
            pos += ciphers_len
            if pos >= len(data):
                return None, None
            comp_methods_len = data[pos]
            pos += 1 + comp_methods_len
            if pos + 2 > len(data):
                return None, None
            ext_total_len = int.from_bytes(data[pos:pos + 2], "big")
```

```
        pos += 2
        ext_types = []
        sni_hostname = None
        alpn = "00"
        versions_found = []
        sig_algs = []
        end_pos = min(pos + ext_total_len, len(data))
        while pos + 4 <= end_pos:
            etype = int.from_bytes(data[pos:pos + 2], "big")
            elen = int.from_bytes(data[pos + 2:pos + 4], "big")
            edata = data[pos + 4:pos + 4 + elen]
            ext_types.append(etype)
            if etype == 0 and len(edata) >= 5:
                s_len = int.from_bytes(edata[3:5], "big")
                if len(edata) >= 5 + s_len:
                    sni_hostname = edata[5:5 + s_len].decode("utf-8", errors="ignore")
            elif etype == 16 and len(edata) >= 3:
                proto_len = edata[2]
                proto = edata[3:3 + proto_len].decode("utf-8", errors="ignore")
                if proto:
                    alpn = proto[0] + proto[-1]
            elif etype == 43 and len(edata) >= 3:
                vlen = edata[0]
                for i in range(1, 1 + vlen, 2):
                    if i + 1 < len(edata):
                        v = int.from_bytes(edata[i:i + 2], "big")
                        versions_found.append(v)
            elif etype == 13 and len(edata) >= 4:
                alg_len = int.from_bytes(edata[0:2], "big")
                for i in range(2, 2 + alg_len, 2):
                    if i + 1 < len(edata):
                        a = int.from_bytes(edata[i:i + 2], "big")
                        sig_algs.append(a)
            pos += 4 + elen
        clean_ciphers = [c for c in ciphers if c not in GREASE_TABLE]
        clean_extensions = [e for e in ext_types if e not in GREASE_TABLE]
        c_count = f"{min(len(clean_ciphers), 99):02d}"
        e_count = f"{min(len(clean_extensions), 99):02d}"
        sni = "d" if 0 in clean_extensions else "i"
        versions = "00"
        if 0x0304 in versions_found:
            versions = "13"
        elif 0x0303 in versions_found:
            versions = "12"
        if versions == "00" and client_version == 0x0303:
            versions = "12"
```

```
        return compute_ja4(clean_ciphers, clean_extensions, versions, sni, alpn,
sni_hostname, sig_algs)
    except Exception:
        return None, None

def compute_ja4(clean_ciphers, clean_extensions, versions, sni, alpn, sni_hostname,
sig_algs=None):
    c_count = f"{min(len(clean_ciphers), 99):02d}"
    e_count = f"{min(len(clean_extensions), 99):02d}"
    prefix = f"t{versions}{sni}{c_count}{e_count}{alpn}"
    hex_ciphers = sorted([f"{c:04x}" for c in clean_ciphers])
    cipher_string = ",".join(hex_ciphers)
    if clean_ciphers:
        cipher_hash = hashlib.sha256(cipher_string.encode()).hexdigest()[:12]
    else:
        cipher_hash = "000000000000"
    ext_for_hash = [e for e in clean_extensions if e not in (0, 16)]
    hex_ext = sorted([f"{e:04x}" for e in ext_for_hash])
    hex_string = ",".join(hex_ext)
    if sig_algs is None:
        sig_algs = []
    hex_algorithms = "_".join([f"{a:04x}" for a in sig_algs])
    if sig_algs:
        raw = hex_string + "_" + hex_algorithms
    else:
        raw = hex_string
    if ext_for_hash:
        ext_hash = hashlib.sha256(raw.encode()).hexdigest()[:12]
    else:
        ext_hash = "000000000000"
    ja4_str = prefix + "_" + cipher_hash + "_" + ext_hash
    print(f"JA4 of Packet is: {ja4_str}")
    return ja4_str, sni_hostname

def process_packet(packet):
    cap = capture(packet)
    if cap is None:
        return
    meta = metadata(packet)
    endpoint_a = (meta["source_ip"], meta["source_port"])
    endpoint_b = (meta["dest_ip"], meta["dest_port"])
    session_key = tuple(sorted([endpoint_a, endpoint_b]))
    ja4_data, sni_hostname = ja4(packet)
    if ja4_data is None:
        raw = try_get_hello_bytes(packet, session_key)
        if raw is not None:
```

```
            ja4_data, sni_hostname = ja4_from_raw(raw)
    if ja4_data is not None:
        session_table[session_key] = (ja4_data, sni_hostname)
        meta["ja4"] = ja4_data
        meta["metadata"]["sni"] = sni_hostname
    else:
        cached = session_table.get(session_key)
        if cached:
            meta["ja4"] = cached[0]
            meta["metadata"]["sni"] = cached[1]
    print(meta)
    return meta

if __name__ == "__main__":
    sniff(iface="eth0", filter="tcp port 443", prn=process_packet, store=0)
```

5. Variables & System Mapping

Variable / Key	Type	Role in `ja4.py`	Destination in Backend / Database / UI
`source_type`	`str`	Always set to `"network"`.	**Backend/DB:** `LogEvent.source_type`. Used by correlation engine to distinguish network logs from host/firewall logs.
`protocol`	`str`	Always set to `"tcp"`.	**Backend/DB:** `LogEvent.protocol`. Filtered in database queries.
`source_ip`	`str`	Client IPv4 or IPv6 address.	**DB & Correlation:** Indexed as `LogEvent.source_ip`. Used to track per-source anomaly scores in sliding windows.
`dest_ip`	`str`	Destination server IP address.	**DB & Threat Intel:** `LogEvent.dest_ip`. Matched against malicious C2 IP blacklists.
`source_port`	`int`	Ephemeral client TCP port.	**DB:** `LogEvent.source_port`. Useful for flow reconstruction.
`dest_port`	`int`	Destination service port (e.g. 443).	**DB:** `LogEvent.dest_port`. Correlated to detect non-standard HTTPS tunneling (e.g. TLS on port 8080).
`tcp_flags`	`str`	TCP control flags (`S`, `SA`, `PA`, `A`, `F`, `R`).	**DB & Engine:** `LogEvent.tcp_flags`. Used to detect SYN flood attacks and half-open port scans.
`packet_size`	`int`	Byte size of raw packet.	**DB & Analytics:** `LogEvent.packet_size`. Used to detect data exfiltration spikes.
`ja4`	`str`	36-char JA4 fingerprint.	**Core Threat Intelligence:** `LogEvent.ja4`. Matched against threat intelligence feeds to identify malware (Cobalt Strike, Metasploit, Sliver) regardless of IP or port. Displayed prominently in React UI.
`metadata.sni`	`str`	Destination domain name extracted from TLS Extension 0.	**DB & Analyst UI:** `LogEvent.metadata["sni"]`. Enables analyst threat hunting

Variable / Key	Type	Role in `ja4.py`	Destination in Backend / Database / UI
			without decrypting packet contents.
`metadata.time_to_live`	`int`	IPv4 TTL or IPv6 Hop Limit.	**DB:** `LogEvent.metadata["time_to_live"]`. Used for OS fingerprinting and detecting routing anomalies.
`session_table`	`dict`	Module-level cache in RAM keyed by sorted endpoints.	**Not sent to Backend.** Purely internal memory to ensure sub-millisecond session lookup.
`hello_buffer`	`dict`	Module-level TCP reassembly buffer.	**Not sent to Backend.** Discarded immediately after record reassembly.